Heuristic approaches

Challenges of this module

* Let us suppose we are facing a problem for which **exact algorithms are impractical**:
  * very short time available
  * very difficult problem
* Can we propose **fast** but **approximate** solution methods?
* **What is the role of the exact methods** seen so far?
* Can we find some **general algorithmic structures** that can be adapted in an easy way to a variety of problems?
* **We have already used these general algorithmic structures**



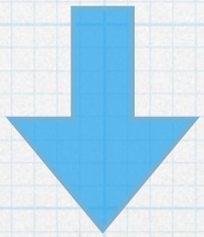
Links with previous knowledge

* Recall the main steps of **Kruskal's algorithm**
* Recall the main steps of the **sorting algorithm** for the investment problem
* **What do they have in common?**
* Recall the main steps of the **augmenting path algorithm**
* Recall the main steps of the **LP algorithm**
* **What do they have in common?**



Facing difficult problems

* When can we say that a problem is "difficult"?
* Find the exact optimal solution
  (potentially in an exponential time)
* Find a good solution in a reasonable time
  (but not provably optimal)



Heuristic algorithms

Exploit the problem structure
Produce good solutions
Fast

Measure the approximation

Consider $P : \min\{c(x) : x \in F\}$

A heuristic algorithm

z_A value of the solution produced by the heuristic

z_{OPT} optimal solution value

Absolute error $z_A - z_{OPT}$

Relative error $\dfrac{z_A - z_{OPT}}{z_{OPT}}$

How to estimate the error without knowing z_{OPT} ?

Use a lower bound

smart relaxations

Approximation schemes

* bounded error
* finite (polynomial) time
* some problems are so difficult that they do not even admit an approximation scheme

Metaheuristics

* Algorithmic paradigms that can be adapted to many problems specifying few elements
* Easy to implement
* Constructive algorithms
  * Greedy
* Improvement algorithms
  * Local search

Other metaheuristics: Graph Optimization

Solution strategy

$$\max \sum_{i=1}^{n} c_i y_i$$

$$\sum_{i=1}^{n} a_i y_i \leq b$$

$$0 \leq y_i \leq 1 \quad i = 1, \ldots, n$$

sort indices so that $i<j \Rightarrow c_i / a_i \geq c_j / a_j$

```
i←1;
repeat
        y_i ← min {1, b/a_i};
        b←b-y_i a_i;
        i←i+1;
until b=0 or i>n
```

Greedy

* Define the **solution building blocks**
* Start from an empty solution
* **Select one element to add**
  (if the resulting **partial solution is feasible**)
* **Example of greedy**
  * Knapsack problem
    * building blocks: objects
    * select the object with best ratio value/weight
    * add the object if the overall weight of the selected objects does not overcome the capacity

Greedy paradigm

E: set of elements
F: family of feasible subsets

```
X ← ∅;          {partial solution}
S ← E;          {set of candidate elements of the solution}
repeat
  e←Best(S);
  S←S\{e};
  if Ind(X, e) then X ←X ∪ {e};
until S = ∅
```

Knapsack

$$\max\{\sum_{i=1}^{n} c_i x_i : \sum_{i=1}^{n} a_i x_i \leq b, x_i \in \{0,1\} \;\; i = 1, \ldots, n\}$$

$$E = \{1, \ldots, n\}$$

F all the subset of objects whose total weight is ≤ b

It is sufficient to specify the rules to recognize a feasible subset

Best(S): $argmax\{\frac{c_i}{a_i}, i \in S\}$

Ind(X, h): $a_h \leq b - \sum_{i \in X} a_i$

Minimum Spanning Tree

Undirected (connected) graph $G = (N, A)$

weights on the arcs $w_{ij} \geq 0 \; \forall (i,j) \in A$

find a tree spanning all nodes A'

minimizing $$\sum_{(i,j) \in A'} w_{ij}$$

$$E = A$$

$F =$ subsets of arcs defining acyclic graphs

Best(S)= $argmin\{w_{ij}, \{i,j\} \in S\}$

Ind(X,{i,j})=if $X \cup \{i,j\}$ is acyclic

In this case the greedy provides always the optimum

This property characterizes the Matroids

Maximum independent set

$G = (N, A)$ undirected graph

$X \subseteq N$ is **independent** if $\forall i, j \in X, \{i, j\} \notin A$

Find the largest independent set

Set Covering

Ground set E

F Family of subsets of E

cost $c_i \geq 0, i \in F$

Find a subfamily $S \subseteq F$

such that each element of E
appears in at least one subset in S

Minimize the sum of the subset costs

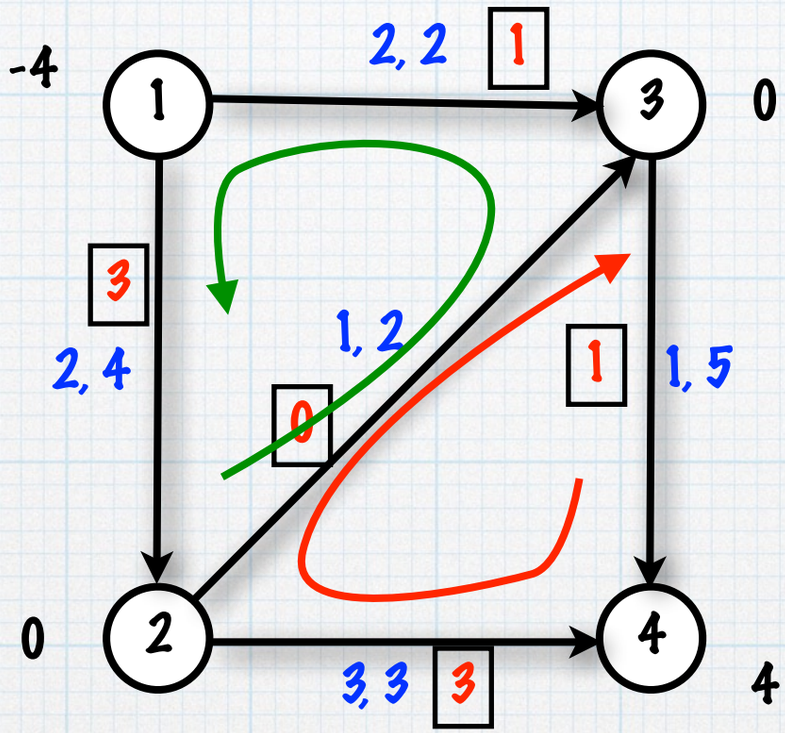
Local Search

* Start with a feasible solution
* Try to improve the current solution by making "small changes"
* Stop if no improvement is found

The obtained solution is a so called "Local Optimum"

Example Minimum Cost Flow

Given a feasible flow





Try to change the solution on cycles

Select one improving change

Other examples of Local Search

* Maximum flow
  * improvement: augmenting path
* Linear Programming
  * improvement: move along a direction

What about shortest paths?

Local Search paradigm

x: feasible solution
N(x): Neighborhood of x {set of feasible solutions "close" to x}

while $\sigma(x) \neq x$ do

    $x \leftarrow \sigma(x)$;

$\sigma(x)$ returns an improving solution in N(x) or x itself if none is found

Tradeoff

Large neighborhood	big improvements few iterations long time to compute $\sigma(x)$
Small neighborhood	fast computation of $\sigma(x)$ many iterations

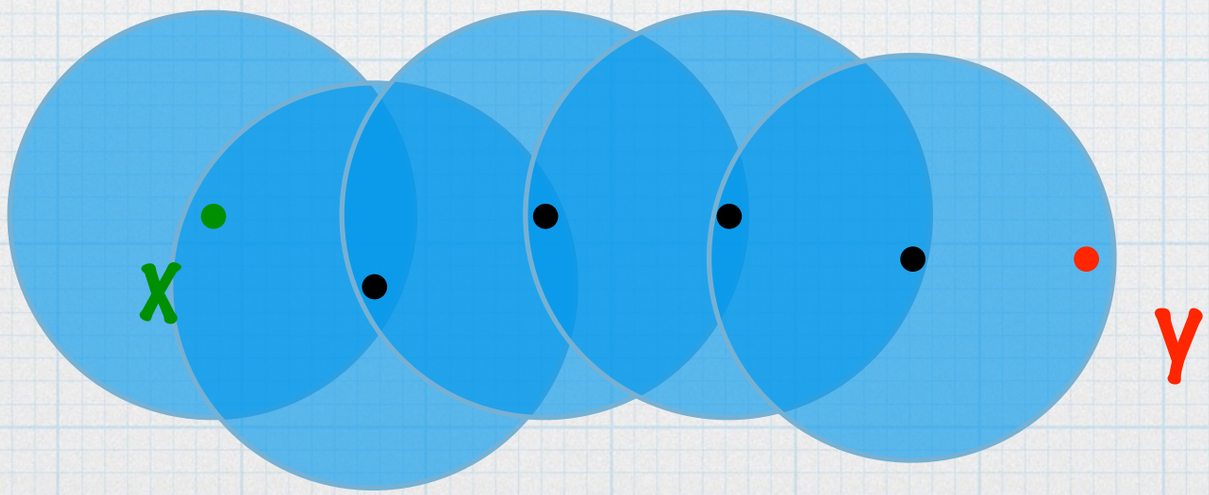
Properties of Neighborhoods

x must belong to N(x)

The search space must be connected

Given two feasible solutions x and y

The definition of N() must be such that it is always possible to migrate from x to y with a sequence of "moves"



Specifying a Local Search

* Define N(x)
* Define $\sigma(x)$
    * best improvement
    * first improvement
    * random selection among the best improvements
    * ...

Max Flow

$$N(x) = \{x' = x \oplus x_p, 0 \le x'_{ij} \le u_{ij}, p \in P_{st}\} \cup \{x\}$$

augmentation of flow

set of paths from s to t

$\sigma(x)$ augmenting path with the minimum number of arcs

Minimum Spanning Tree

TSP

Resources

* Section 5



Additional material

* E. Aarts J.K. Lenstra
  "Local Search in Combinatorial Optimization"



This is the end...

	prescribed	actual
lectures	32	35
exercises	13	17
lab	4	8

bonus questions: 8

graded answers: >1300

mini lab projects:

Kahoot: 6

peer review: 145

blended activities: 3

Intended Learning Objectives

* Understand the mathematical language of optimization models and some important classes of models
* Create optimization models starting from the description of the problems
* Recall the main properties and algorithms (including the worst case complexity) for minimum cost spanning tree, shortest path, maximum flow
* Apply graph optimization algorithms to simple instances
* Create optimization models based on graph problems
* The geometrical and algebraic aspects of Linear Programming (LP), and an LP algorithm
* LP duality and the complementary slackness optimality conditions
* The economical interpretation of the dual of some LP problems
* The cutting plane method with fractional Gomory cuts for ILP
* The Branch-and-Bound method for ILP and other combinatorial optimization problem
* Build optimization models starting from the decision problem descriptions
* Identify an appropriate graph optimization algorithm to tackle a given decision problem and apply it to small examples
* Construct the dual of LP problems in explicit and parametric form
* Identify the characteristics of the Greedy and Local Search heuristic algorithms
* Create simple heuristics for optimization problems